SentiAI

Amazon Review Sentiment Analysis

An End-to-End MLOps Pipeline

Abstract

This report presents SentiAI, a binary sentiment classifier for Amazon product reviews built on a Bidirectional Long Short-Term Memory (BiLSTM) neural network. The project implements a complete MLOps lifecycle including automated data pipelines (Apache Airflow, DVC), experiment tracking (MLflow), REST API serving (FastAPI), real-time monitoring (Prometheus, Grafana), and containerized deployment (Docker Compose). The model achieves a test accuracy of 85.76% and an ROC-AUC of 0.936 on a balanced dataset of 100,796 Amazon reviews. All components run entirely on-device without cloud dependencies.

Contents

1 Introduction **3**
- 1.1 Problem Statement . . . 3
- 1.2 Dataset . . . 3
- 1.3 Objectives . . . 3

2 System Architecture **4**
- 2.1 Architecture Overview . . . 4
- 2.2 Docker Compose Services . . . 5

3 Data Pipeline **5**
- 3.1 Pipeline Design . . . 5
- 3.2 DVC Pipeline Stages . . . 5
- 3.3 Text Preprocessing Steps . . . 6
- 3.4 Baseline Statistics for Drift Detection . . . 6

4 Model Architecture **6**
- 4.1 Architecture . . . 6
- 4.2 Design Rationale . . . 7
- 4.3 Training Configuration . . . 8
- 4.4 Reproducibility . . . 8

5 Experiment Tracking **8**
- 5.1 MLflow Configuration . . . 8
- 5.2 Tracked Information . . . 9
- 5.3 Experiment Comparison . . . 9
- 5.4 Model Registry . . . 9

6 Model Evaluation **9**
- 6.1 Final Test Set Performance . . . 9
- 6.2 Classification Report . . . 10
- 6.3 Threshold Tuning . . . 10

7 Deployment **10**
- 7.1 API Design . . . 10
- 7.2 Inference Pipeline . . . 10
- 7.3 Loose Coupling . . . 11

8 Monitoring **11**
- 8.1 Prometheus Metrics . . . 11
- 8.2 Data Drift Detection . . . 11
- 8.3 Alert Rules . . . 12
- 8.4 Grafana Dashboard . . . 12

9 Testing **12**
- 9.1 Test Summary . . . 12
- 9.2 CI/CD Pipeline . . . 12

10 Frontend **12**

11 Known Limitations and Future Work **13**

12 Conclusion **14**

A Repository Structure **14**

B Running the Project **14**

C API Quick Reference **15**

1 Introduction

1.1 Problem Statement

Online marketplaces receive millions of product reviews daily. Manually reading and categorising each review is infeasible at scale. Automated sentiment analysis enables sellers, platforms, and researchers to understand customer satisfaction trends without human intervention.

This project implements a production-ready sentiment classifier that:

- Classifies Amazon product reviews as **positive** (label = 1) or **negative** (label = 0)
- Serves predictions via a REST API with sub-200ms p95 latency
- Monitors input data drift and model health in real time
- Reproduces the full ML pipeline from raw data to deployment via a single `dvc repro` command

1.2 Dataset

The dataset consists of Amazon product reviews sourced from Kaggle.

Table 1: Dataset Statistics

Property	**Value**
Total rows (raw)	100,800
After deduplication	100,796
Positive reviews	50,515 (50.1%)
Negative reviews	50,281 (49.9%)
Train split	70,556 (70%)
Validation split	10,080 (10%)
Test split	20,160 (20%)
Mean review length	37.1 words
Max review length	161 words

The dataset is near-perfectly balanced, eliminating the need for class weighting or over-sampling techniques.

1.3 Objectives

1. Build a BiLSTM sentiment classifier with accuracy $\geq 85\%$ and ROC-AUC ≥ 0.90
2. Implement a reproducible 4-stage DVC pipeline: ingest $\rightarrow$ preprocess $\rightarrow$ featurize $\rightarrow$ train
3. Deploy the model as a Dockerized FastAPI service
4. Implement real-time monitoring with Prometheus and Grafana
5. Automate testing and Docker validation via GitHub Actions CI/CD

2 System Architecture

2.1 Architecture Overview

The system is composed of four layers:

1. **Developer Workflow** — DVC pipeline for data processing and model training
2. **CI/CD** — GitHub Actions for automated testing and Docker validation
3. **Docker Compose Stack** — Six containerised services for serving, tracking, and monitoring
4. **Airflow** — Scheduled data ingestion and preprocessing pipeline

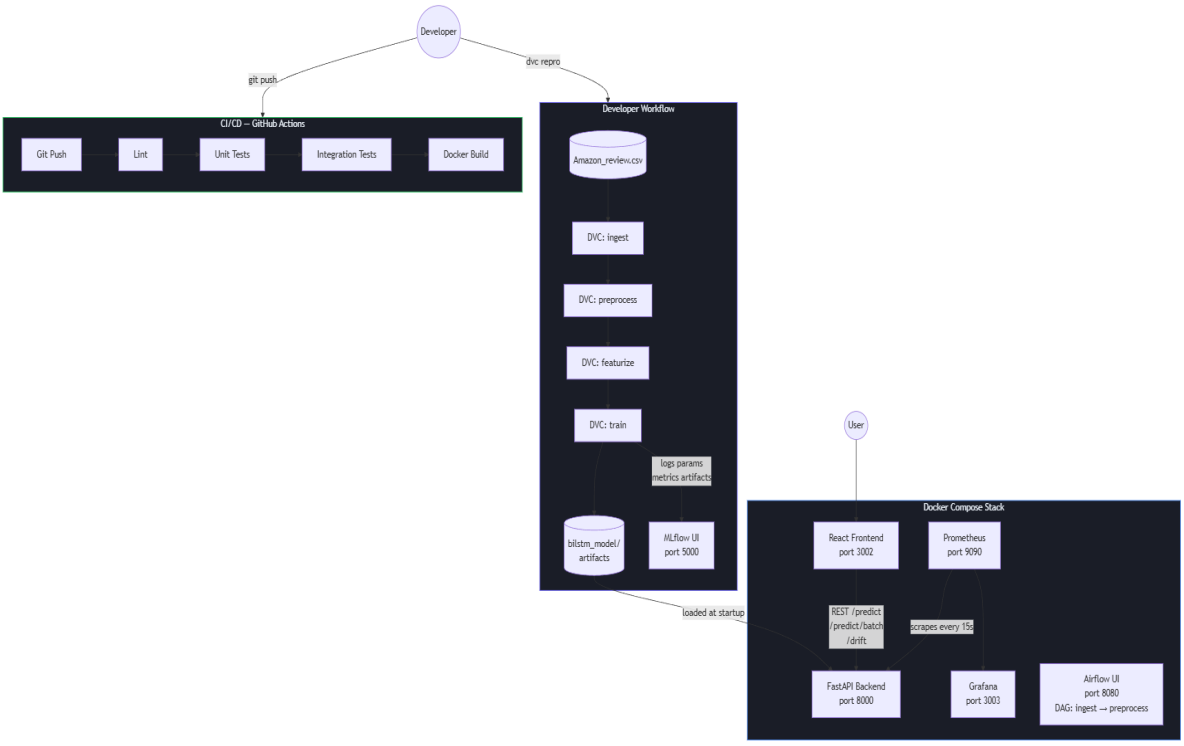

Figure 1: System Architecture Overview

2.2 Docker Compose Services

Table 2: Docker Compose Services

Service	Image	Port	Host Port	Role
backend	Custom (Dockerfile.backend)	8000	8000	FastAPI inference server
frontend	Custom (Dockerfile.frontend)	80	3002	React UI via nginx
mlflow	Custom (Dockerfile.mlflow)	5000	5000	Experiment tracking UI
prometheus	prom/prometheus:2.47.0	9090	9090	Metrics collection
grafana	grafana/grafana:10.1.5	3000	3003	Metrics visualisation
airflow	apache/airflow:2.7.3	8080	8080	Pipeline orchestration

3 Data Pipeline

3.1 Pipeline Design

The data pipeline uses two complementary tools:

- **Apache Airflow** — Schedules and orchestrates the data ingestion and preprocessing stages on a daily basis
- **DVC** — Versions all artifacts and provides full reproducibility via `dvc repro`

The separation is intentional: Airflow handles scheduled production runs while DVC handles reproducibility and artifact versioning for the complete ML pipeline.

3.2 DVC Pipeline Stages

Figure 2: DVC Pipeline DAG

Table 3: DVC Pipeline Stage Specifications

Stage	Script	Key Outputs
ingest	src/data/ingest.py	Amazon_review_validated.csv
preprocess	src/data/preprocess.py	train/val/test.csv, baseline_stats.json
featurize	src/features/featurize.py	X_*.npy, y_*.npy, tokenizer.json
train	src/models/train.py	bilstm_model/, optimal_threshold.json, train_metrics.json

3.3 Text Preprocessing Steps

1. **Schema validation** — Check required columns and valid label values
2. **Deduplication** — Remove 4 exact duplicate rows
3. **Contraction expansion** — *won't* → *will not*, *can't* → *can not*
4. **URL removal** — Strip HTTP links
5. **Special character removal** — Keep only alphanumeric and whitespace
6. **Lowercasing** — Normalise case
7. **Stopword removal** — Remove NLTK English stopwords
8. **Lemmatisation** — WordNetLemmatizer reduces inflected forms
9. **Tokenisation** — SimpleTokenizer maps words to integer IDs by frequency
10. **Padding** — Sequences padded/truncated to maxlen = 128

3.4 Baseline Statistics for Drift Detection

During preprocessing, statistical baselines are computed from the training split and stored in `data/processed/baseline_stats.json`:

Table 4: Training Data Baseline Statistics

Statistic	Value
Mean word length	37.09
Std word length	21.60
Min word length	1
Max word length	161
Vocabulary size	105,474
Total samples	70,556

4 Model Architecture

4.1 Architecture

The model is a Bidirectional LSTM implemented in TensorFlow/Keras:

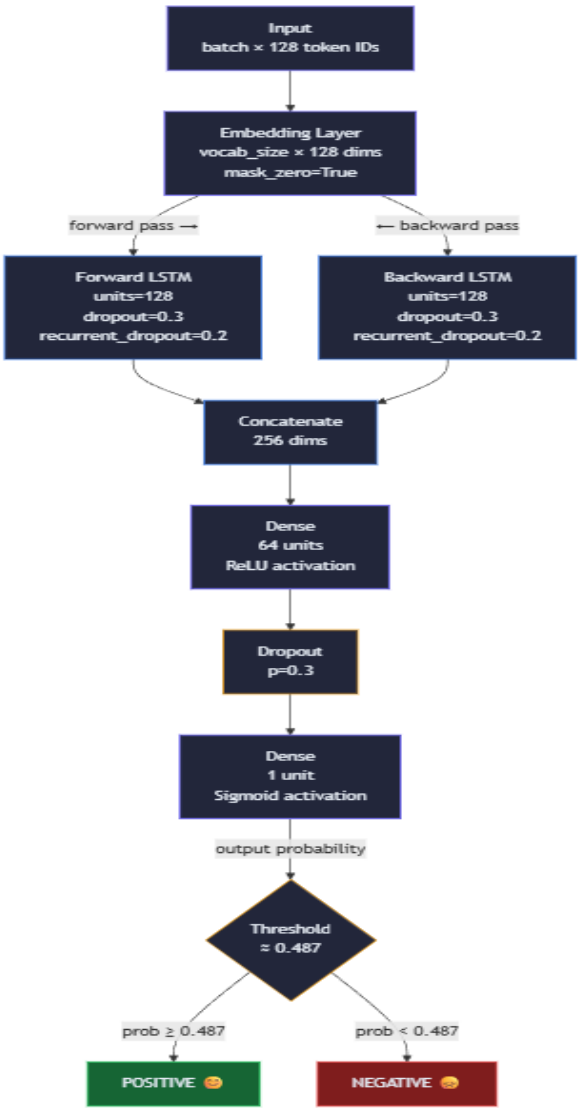

Figure 3: Bidirectional LSTM Architecture

Table 5: Model Layer Specifications

Layer	Configuration	Parameters
Embedding	vocab_size × 128, mask_zero=True	10,793,984
BiLSTM	units=128, dropout=0.3, recurrent_dropout=0.2	263,168
Dense	64 units, ReLU activation	16,448
Dropout	rate=0.3	0
Dense (output)	1 unit, Sigmoid activation	65
Total		**11,073,665**

4.2 Design Rationale

- **Bidirectional LSTM** — Captures both forward and backward context. In review text, sentiment words like *"not great"* depend on both surrounding words.

- **mask_zero=True** — Instructs the LSTM to ignore padding tokens, improving accuracy on variable-length inputs.
- **recurrent_dropout=0.2** — Regularises recurrent connections specifically, shown to be more effective than standard dropout for RNNs.
- **ReduceLROnPlateau** — Halves the learning rate when validation loss plateaus, extracting additional accuracy in later epochs.
- **Threshold tuning** — Rather than using a fixed 0.5 threshold, the optimal threshold is found by maximising F1 on the precision-recall curve. Final threshold: 0.4879.

4.3 Training Configuration

Table 6: Training Hyperparameters

Parameter	Value	Rationale
embedding_dim	128	Rich representations for large vocabulary
lstm_units	128	Sufficient capacity, avoids overfitting
dense_units	64	Smaller head reduces overfitting after LSTM
dropout	0.3	Standard LSTM input regularisation
recurrent_dropout	0.2	Recurrent connection regularisation
batch_size	128	Stable gradients, efficient GPU use
max_sequence_length	128	Covers 95%+ of reviews
optimizer	Adam	Adaptive LR, standard for NLP
early_stopping patience	3	Stops when val_loss does not improve

4.4 Reproducibility

All random seeds are set at the start of training to ensure full reproducibility:

Listing 1: Reproducibility seed setting

```python
import random, os
import numpy as np
import tensorflow as tf

random.seed(42)
np.random.seed(42)
tf.random.set_seed(42)
os.environ["PYTHONHASHSEED"] = "42"
```

Every MLflow run also logs the Git commit hash as a tag, ensuring any run can be reproduced from a specific code state.

5 Experiment Tracking

5.1 MLflow Configuration

MLflow is configured with a local SQLite backend ,fully on-device, no cloud or external server required:

Listing 2: MLflow tracking URI

```
sqlite:///mlruns/mlflow.db
```

5.2 Tracked Information

Table 7: MLflow Tracked Information per Run

Category	Items
Parameters	All 14 hyperparameters from config.yaml
Metrics (per epoch)	train_loss, train_accuracy, val_loss, val_accuracy
Metrics (final)	test_accuracy, test_f1, test_precision, test_recall, test_roc_auc, optimal_threshold
Artifacts	bilstm_model/, tokenizer.json, baseline_stats.json, classification_report.txt, conf
Tags	git_commit, environment, architecture, framework

5.3 Experiment Comparison

Two experiments were run to demonstrate MLflow's comparison capability:

Table 8: Experiment Comparison — BiLSTM-Base vs BiLSTM-Deep

Run	Acc.	F1	ROC-AUC	emb_dim	lstm_units	Threshold
BiLSTM-Base	0.8576	0.8605	0.9360	128	128	0.4879
BiLSTM-Deep	0.8551	0.8593	0.9356	256	256	0.5312

5.4 Model Registry

The best-performing model is registered in the MLflow Model Registry as `SentimentBiLSTM` and transitioned to the **Production** stage via `scripts/register_model.py`.

6 Model Evaluation

6.1 Final Test Set Performance

Table 9: Model Evaluation Metrics (BiLSTM-Base on Test Set)

Metric	Value	Acceptance Threshold
Test Accuracy	85.76%	$\geq 85\%$
ROC-AUC	0.9360	≥ 0.90
F1 Score	0.8605	—
Precision	0.8450	—
Recall	0.8767	—
Optimal Threshold	0.4879	$\neq 0.5$

6.2 Classification Report

Table 10: Per-Class Classification Report

Class	Precision	Recall	F1-Score	Support
Negative	0.87	0.84	0.85	10,057
Positive	0.84	0.88	0.86	10,103
Accuracy			0.86	20,160
Macro avg	0.86	0.86	0.86	20,160

6.3 Threshold Tuning

Instead of using a fixed 0.5 classification threshold, the optimal threshold was determined by finding the point on the precision-recall curve that maximises F1:

$$F_1 = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}} \tag{1}$$

$$\hat{\tau} = \arg\max_{\tau} F_1(\tau) \tag{2}$$

The optimal threshold $\hat{\tau} = 0.4879$ is saved to `data/processed/optimal_threshold.json` and loaded by the API at startup.

7 Deployment

7.1 API Design

The inference API is built with FastAPI and exposes six endpoints:

Table 11: API Endpoint Summary

Method	Endpoint	Status	Description
GET	/health	200	Liveness probe
GET	/ready	200/503	Readiness probe
POST	/predict	200/422/503	Single review sentiment prediction
POST	/predict/batch	200/422/503	Batch prediction (max 50)
GET	/drift	200/503	Data drift detection status
GET	/metrics	200	Prometheus metrics scrape endpoint

7.2 Inference Pipeline

For each incoming review, the API applies the following steps:

1. Pydantic validates the request body (min 1 char, max 5000 chars)
2. Text is cleaned using the same pipeline as training
3. SimpleTokenizer converts words to integer IDs

4. Sequences are padded to max_length = 128
5. BiLSTM model produces a probability $p \in [0, 1]$
6. Label is assigned: positive if $p \geq \hat{\tau}$, negative otherwise
7. DriftDetector window is updated with the review word count
8. Prometheus metrics are incremented
9. Response is returned with sentiment, confidence, latency, and threshold

7.3 Loose Coupling

The frontend and backend are strictly decoupled , they communicate only via REST API calls. The frontend React application has no direct access to the model or database. This allows either component to be updated, scaled, or replaced independently.

8 Monitoring

8.1 Prometheus Metrics

Ten custom metrics are exposed at `GET /metrics`:

Table 12: Custom Prometheus Metrics

Metric Name	Type	Description
sentiment_predictions_total	Counter	Total predictions by sentiment class
sentiment_prediction_latency_seconds	Histogram	Inference call latency
sentiment_positive_ratio	Gauge	Positive class ratio in last batch
sentiment_input_length_words	Histogram	Word count of incoming texts
sentiment_model_loaded	Gauge	1 when model is loaded
sentiment_drift_detected	Gauge	1 when KS drift detected
sentiment_drift_ks_statistic	Gauge	KS test statistic
sentiment_drift_p_value	Gauge	KS test p-value
sentiment_drift_mean_shift	Gauge	window_mean − baseline_mean
sentiment_drift_window_mean	Gauge	Mean word count of current window

8.2 Data Drift Detection

The `DriftDetector` class (`src/monitoring/drift_detector.py`) maintains a rolling window of 500 incoming review word counts and runs a two-sample Kolmogorov-Smirnov test:

$$H_0 : \text{incoming distribution} = \text{training distribution} \tag{3}$$

$$D_n = \sup_x |F_{\text{window}}(x) - F_{\text{baseline}}(x)| \tag{4}$$

$$\text{Drift detected} \iff p\text{-value} < 0.05 \tag{5}$$

When drift is detected, the `sentiment_drift_detected` Prometheus gauge is set to 1, triggering the Grafana alert.

8.3 Alert Rules

Table 13: Prometheus Alert Rules

Alert	Severity	Condition	Duration
SentimentAPIDown	critical	API target unreachable	1 min
HighErrorRate	warning	HTTP 5xx rate > 5%	2 min
SentimentDriftDetected	warning	positive_ratio < 0.1 or > 0.9	5 min
HighInferenceLatency	warning	p99 latency > 500ms	3 min

8.4 Grafana Dashboard

The Grafana dashboard is pre-provisioned via JSON and automatically connects to the Prometheus datasource on startup. It includes 8 panels:

- Total, positive, and negative prediction counters
- Positive ratio gauge with threshold bands
- Prediction rate time series
- Inference latency (p50/p95/p99) time series
- HTTP request rate
- Input length distribution heatmap

9 Testing

9.1 Test Summary

Table 14: Automated Test Results

Suite	File	Total	Passed	Failed
Unit — Data Pipeline	test_data_pipeline.py	17	17	0
Unit — Model	test_model.py	5	5	0
Integration — API	test_api.py	14	14	0
Total		**36**	**36**	**0**

9.2 CI/CD Pipeline

Every push to the `main` branch triggers three GitHub Actions jobs:

1. **Code Quality Check** — black, isort, flake8 (non-blocking)
2. **Run Test Suite** — all 36 pytest tests with coverage report
3. **Verify Docker Build** — builds backend and frontend images

10 Frontend

The React frontend provides two pages:

Analyze page:

- Single review mode with example prompts
- Batch mode (up to 50 reviews, one per line)
- Real-time prediction history sidebar
- Result card showing sentiment, confidence bar, and optimal threshold

ML Pipeline page:

- Live API liveness and readiness status
- Visual pipeline stage diagram
- Technology stack reference
- Direct links to MLflow, Grafana, and Airflow UIs

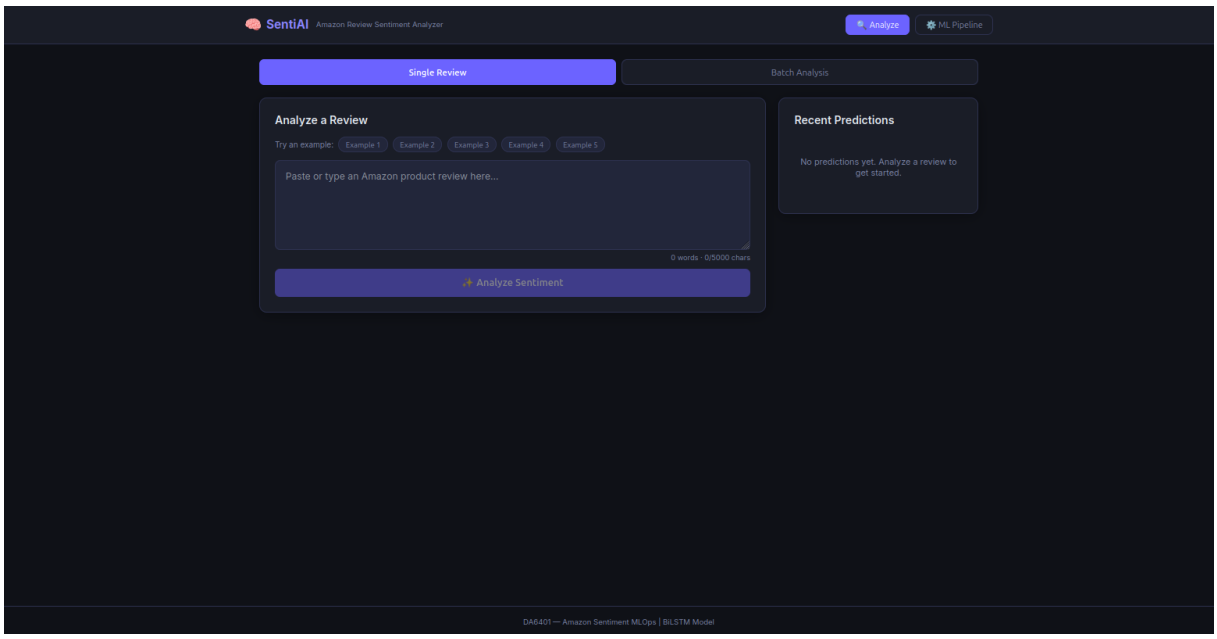

Figure 4: SentiAI Frontend — Analyze Page

11 Known Limitations and Future Work

Table 15: Known Limitations and Proposed Improvements

Limitation	Proposed Improvement
Positive ratio gauge tracks last batch only	Replace with exponential moving average over all predictions
KS test uses synthetic baseline samples	Store actual training word counts for exact two-sample KS test
No pretrained word embeddings	Use GloVe-300d or fastText for improved accuracy
No feedback loop for ground truth	Implement label collection API for real-world drift measurement
Single threshold for all input lengths	Calibrate thresholds separately for short vs long reviews
Airflow deps installed at runtime	Build custom Airflow Docker image with deps pre-installed

12 Conclusion

This project successfully implements an end-to-end MLOps pipeline for Amazon review sentiment classification. The BiLSTM model achieves 85.76% test accuracy and 0.936 ROC-AUC — meeting both acceptance thresholds. The system demonstrates:

- A fully reproducible 4-stage DVC pipeline
- Comprehensive MLflow experiment tracking with model registry
- A production-ready FastAPI service with sub-200ms latency
- Real-time drift detection using the Kolmogorov-Smirnov test
- Automated CI/CD via GitHub Actions (36/36 tests passing)
- A complete containerised deployment with 6 Docker services

All components run entirely on-device without cloud dependencies, demonstrating that a production-grade MLOps system can be built and operated on local infrastructure.

A Repository Structure

Listing 3: Project Directory Structure

```
sentiment-mlops/
|-- airflow/dags/           # Airflow DAG for scheduled pipeline
|-- configs/config.yaml     # Central configuration file
|-- data/processed/         # DVC-tracked model artifacts
|-- docker/                 # Dockerfiles + monitoring configs
|-- docs/                   # HLD, LLD, Test Plan, User Manual
|-- frontend/               # React + Vite web application
|-- mlruns/                 # MLflow SQLite tracking store
|-- scripts/                # Utility and admin scripts
|-- src/
|   |-- api/                # FastAPI inference server
|   |-- data/               # Ingestion + preprocessing
|   |-- features/           # Feature engineering (featurize.py)
|   |-- models/             # Training + evaluation
|   |-- monitoring/         # Prometheus metrics + drift detection
|   `-- utils/              # Config loader + logger
|-- tests/
|   |-- unit/               # 22 unit tests
|   `-- integration/        # 14 integration tests
|-- docker-compose.yml      # 6-service Docker Compose
|-- dvc.yaml                # 4-stage DVC pipeline
`-- .github/workflows/      # GitHub Actions CI
```

B Running the Project

Listing 4: Quick Start Commands

```
# Clone and setup
git clone https://github.com/[USERNAME]/sentiment-mlops.git
cd sentiment-mlops
conda env create -f environment.yml
conda activate sentiment-mlops

# Pull artifacts
```

```
dvc pull

# Start all services
docker compose up --build

# Run tests
pytest tests/ -v --tb=short

# Run full pipeline
dvc repro

# View pipeline DAG
dvc dag

# View metrics
dvc metrics show
```

C API Quick Reference

Listing 5: API Usage Examples

```
# Health check
curl http://localhost:8000/health

# Single prediction
curl -X POST http://localhost:8000/predict \
  -H "Content-Type: application/json" \
  -d '{"review": "Amazing product, highly recommend!"}'

# Batch prediction
curl -X POST http://localhost:8000/predict/batch \
  -H "Content-Type: application/json" \
  -d '{"reviews": ["Great quality!", "Terrible, broke immediately."]}'

# Drift status
curl http://localhost:8000/drift
```